MINISTERUL EDUCAȚIEI ȘI CERCETĂRII

UNIVERSITATEA TEHNICĂ DIN CLUJ-NAPOCA

Faculty of Automation and Computer Science

Active Suspension

Project documentation — System Identification

Name: POP Raul
Specialization: Automation and Applied Informatics
Group: 30135

Introduction

The purpose of this work is to identify a second-order system with complex conjugate poles, starting from experimental data provided as input and output vectors. The data were acquired through a Simulink model, the system having a step-type input signal.

From the graphical representation of the response, the behavior typical of a second-order system can be observed, starting with zero slope at the moment the input signal switches.

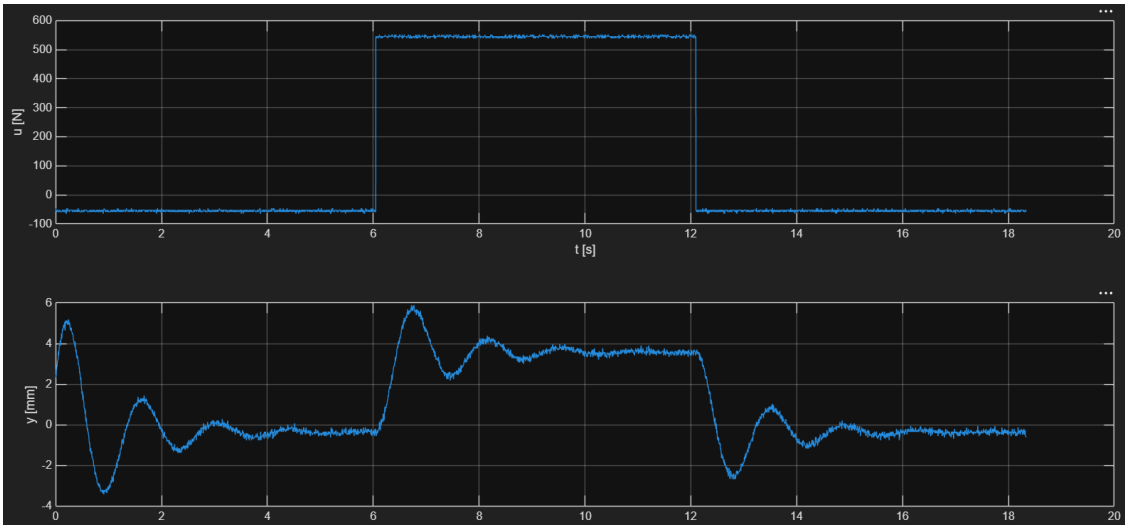


Figure 1 — Acquired signals: input u(t) and output y(t).

A suitable structure for this type of response is the second-order transfer function with complex conjugate poles:

$$\mathrm{H}(s) = \frac{K{\omega_n}^2}{s^2 + 2\zeta\omega_n s + {\omega_n}^2}$$

The identification problem is therefore reduced to determining three parameters: the proportionality constant K, the natural frequency ωn and the damping ratio ζ. These are determined in the next section using the linear regression method.

1. Linear Regression Method

1.1 Theoretical background

The linear regression method consists in building a cost function whose minimum gives the best approximation of a data set by a straight line, so that the resulting error is minimal. The cost function is the sum of squared errors, which removes the influence of the error sign on the result:

$$J = \Sigma\,(y - \widehat{y})^2 = \Sigma(y - \alpha t - \beta)^2$$

where the coefficients to be determined are α and β. For a minimum cost, it is sufficient that the partial derivatives of J with respect to each parameter are equal to zero:

$$\frac{\partial J}{\partial \alpha} \;=\; \alpha \cdot \Sigma t^2 \;+\; \beta \cdot \Sigma t \;-\; \Sigma(t \cdot y) \;=\; 0$$

$$\frac{\partial J}{\partial \beta} = \alpha \cdot \Sigma t + \beta \cdot \Sigma 1 - \Sigma y = 0$$

After rearranging, the system of equations can be written in matrix form:

$$\begin{bmatrix} \Sigma t^2 & \Sigma t \\ \Sigma t & N \end{bmatrix} \cdot \begin{bmatrix} \alpha \\ \beta \end{bmatrix} = \begin{bmatrix} \Sigma (t \cdot y) \\ \Sigma y \end{bmatrix}$$

or, equivalently:

$$\begin{bmatrix} \alpha \\ \beta \end{bmatrix} = \begin{bmatrix} \Sigma t^2 & \Sigma t \\ \Sigma t & N \end{bmatrix}^{-1} \cdot \begin{bmatrix} \Sigma (t \cdot y) \\ \Sigma y \end{bmatrix}$$

1.2 Determining the proportionality constant K

The parameter K, also called the proportionality constant, is determined as the ratio between the output variation and the input variation, computed in steady state:

$$K = \frac{(y_{\text{st}} - y_0)}{(u_{\text{st}} - u_0)}$$

The values u0, y0, ust and yst were computed as means of the signals over intervals chosen in the steady state, before and after the step switch:

```
i1 = 2461;  i2 = 2907;  i3 = 5482;  i4 = 5979;

u0  = mean(u(i1:i2));   y0  = mean(y(i1:i2));
ust = mean(u(i3:i4));   yst = mean(y(i3:i4));

K = (yst - y0) / (ust - u0);
```

1.3 Determining the parameters ζ and ωn

Determining the other two parameters requires a sequence of steps. Starting from the step response of the second-order system:

$$y(t) = \mathrm{L}^{-1}\left(\mathrm{H}(s) \cdot 1/s\right)(t) = y_0 + (y_{\text{st}} - y_0)\left[1 - \frac{\mathrm{e}^{-\zeta\omega_n(t-t_0)}}{\sqrt{1-\zeta^2}} \cdot \sin\left(\omega_n\sqrt{1-\zeta^2}\,(t-t_0) + \varphi\right)\right]$$

$$|y(t) - y_{st}| = \frac{y_{st} - y_0}{\sqrt{1-\zeta^2}} e^{\mathrm{Re}\{s_{1,2}\}(t-t_0)} \left|\sin\left(Im\left\{s_{1,2}\right\}\right)(t-t_0) + \varnothing\right|$$

by rewriting and taking the logarithm, a linear relation is obtained:

$$\ln\left(y(t) - y_{\text{st}}\right) = \alpha \cdot t + C, \quad where \quad \alpha = \mathrm{Re}\left\{s_{1,2}\right\} = -\zeta\omega_n$$

This is, in fact, the equation of a straight line, y = a·t + b. To visualize the quantity y = |y(t) − yst|, the values are chosen from the transient section, close enough to settling — but no closer than 1–2%, otherwise taking the logarithm would introduce values that distort the regression line. By subtracting the steady-state value from the transient section and taking the absolute value, only the “jumps” remain, that is, the maxima and minima of the damped oscillation.

```
tmin = 3033;  tmax = 6072;
y_aux = abs(y(tmin:tmax) - yst);
t_aux = t(tmin:tmax);
```

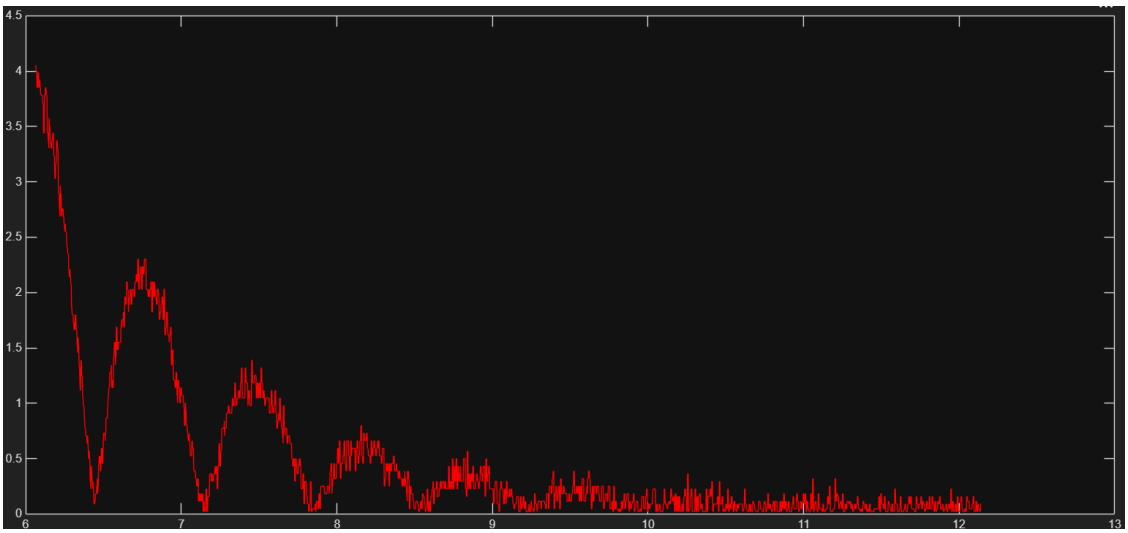


Figure 2 — Section of the damped oscillations, |y(t) − yst|.

In practice at least 2 points are needed; for this project 5 were extracted, and based on them, after taking the logarithm, the regression line that best fits the data set was drawn.

```
im1 = 6;  im2 = 343;  im3 = 710;  im4 = 1070;  im5 = 1402;

t_reg = t_aux([im1 im2 im3 im4 im5]);
y_reg = log(y_aux([im1 im2 im3 im4 im5]));
```

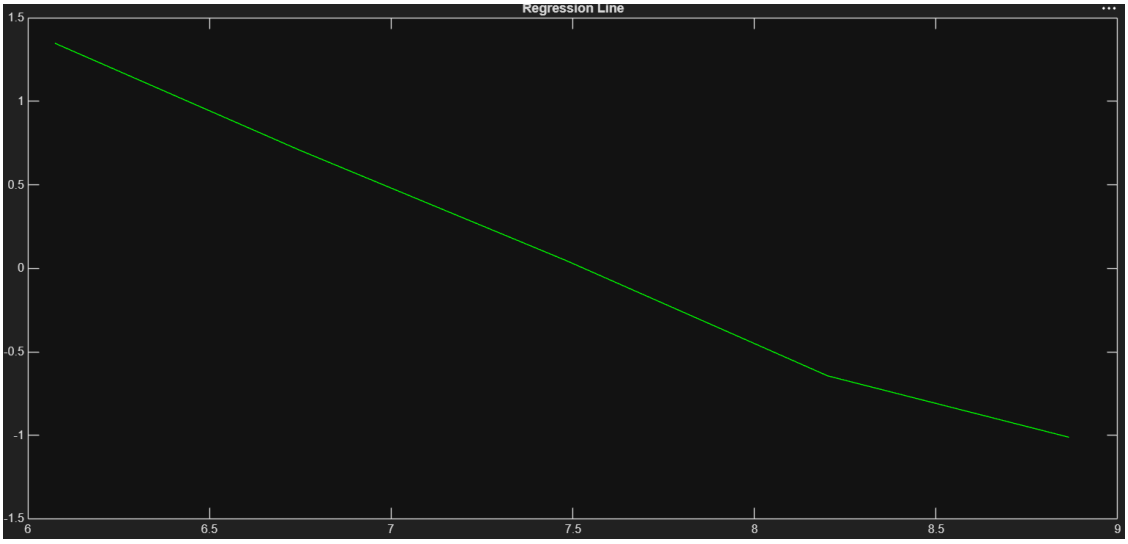


Figure 3 — Regression line.

The change of slope observed towards the end of the plot is caused strictly by the signal noise: approaching the steady-state value leads to taking the logarithm of very small values, strongly affected by noise.

The goal is to determine ωn and ζ, based on the relations:

$$\zeta = \frac{-\mathrm{Re}\{s_{1,2}\}}{\sqrt{\mathrm{Re}\{s_{1,2}\}^2 + \mathrm{Im}\{s_{1,2}\}^2}}$$

$$\omega_n = \frac{-\mathrm{Re}\{s_{1,2}\}}{\zeta}, \quad for \ \zeta \in (-1,\ 1), \ \ \zeta \neq 0$$

As mentioned, the real part is α, determined from the regression. The imaginary part can be obtained directly from the oscillation frequency:

$$\mathrm{Im}\{s_{1,2}\} = \omega_{\mathrm{osc}} = \frac{2\pi}{\mathrm{T}_{\mathrm{osc}}}$$

and Tosc is determined directly as the difference between two successive maxima of the damped oscillation.

1.4 Model reconstruction

Once all parameters (K, ζ, ωn) are determined, the model can be reconstructed in the two standard forms: transfer function and state space.

Transfer function

```
H = tf(K*wn^2, [1, 2*zeta*wn, wn^2]);
y_sim_tf = lsim(H, u, t);
```

The simulation yields the estimated signal, using the lsim function. This simulation starts from zero initial conditions; it takes some time until the simulated response approaches the real one, which is why the errors J and eps_MPN will be considerably larger than in the state-space simulation (section 1.5).

State space

$$\mathrm{A} = \begin{bmatrix} 0 & 1 \\ -{\omega_n}^2 & -2\zeta\omega_n \end{bmatrix};$$

$$\mathrm{B} = \begin{bmatrix} 0 \\ K{\omega_n}^2 \end{bmatrix};$$

$$\mathrm{C} = \begin{bmatrix} 1 & 0 \end{bmatrix}; \mathrm{D} = 0;$$

```
sys = ss(A, B, C, D);
y_sim_ss = lsim(sys, u, t, [y(1) 24.49]);
```

For the initial conditions, the starting position is known directly from the measured signal. To determine the initial velocity there are two approaches: an analytical one (differentiating the signal and extracting its first value) and a numerical one — the interval halving (bisection) method, used in this project. Through successive calibration the value 24.49 was reached, beyond which any other change worsened the validation error.

1.5 Model validation

To verify the proposed models, the two performance indicators presented in the System Identification course were computed:

$$J = \frac{\| \varepsilon \|_2}{\sqrt{N}}$$

$$\varepsilon_{\mathrm{MPN}} = \frac{\| \varepsilon \|_2}{\| y - \bar{y} \|_2}$$

To further reduce the error, the measured output signal was filtered both with a 4th-order Butterworth filter and with a moving average filter.

State-space model

The state-space model follows the measured signal very well, both raw and filtered. After Butterworth filtering, the error reached eps_MPN = 1.25%, below the imposed 10% threshold.

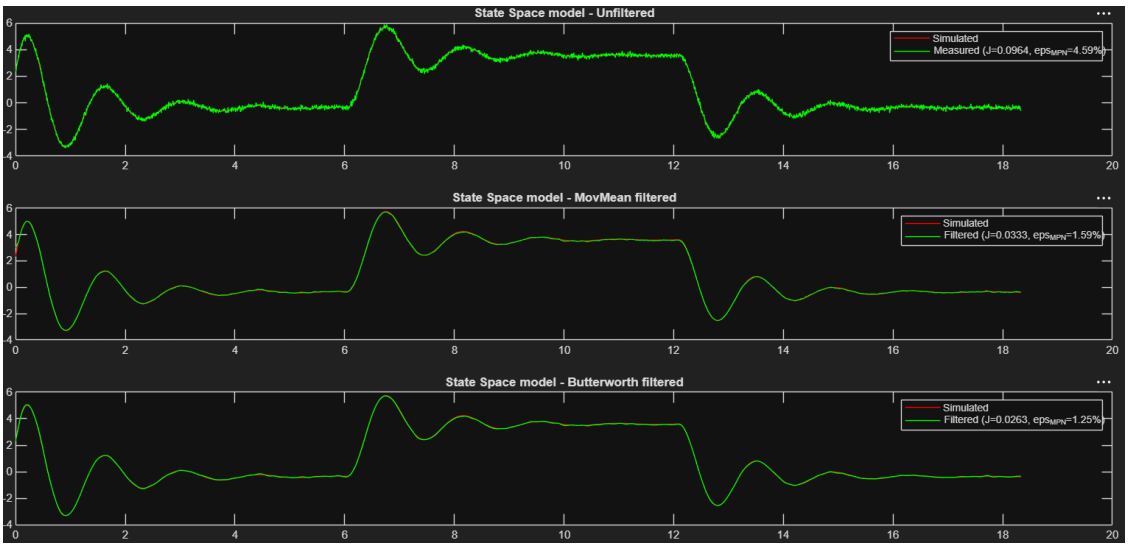


Figure 4 — Validation of the state-space model.

Transfer function model

The transfer function model shows a significantly larger error (eps_MPN ≈ 40%), although the parameters K, ζ and ωn are identical to those used in the state-space model. The cause is the lack of initial conditions in the lsim call for the transfer function — unlike the state-space simulation, where the initial condition was specified explicitly ([y(1) 24.49]), the transfer-function simulation starts by default from zero initial conditions. The large error does not persist on test intervals not previously used for calibration, which confirms that it is not a defect of the chosen data section, but the direct effect of the zero initial condition.

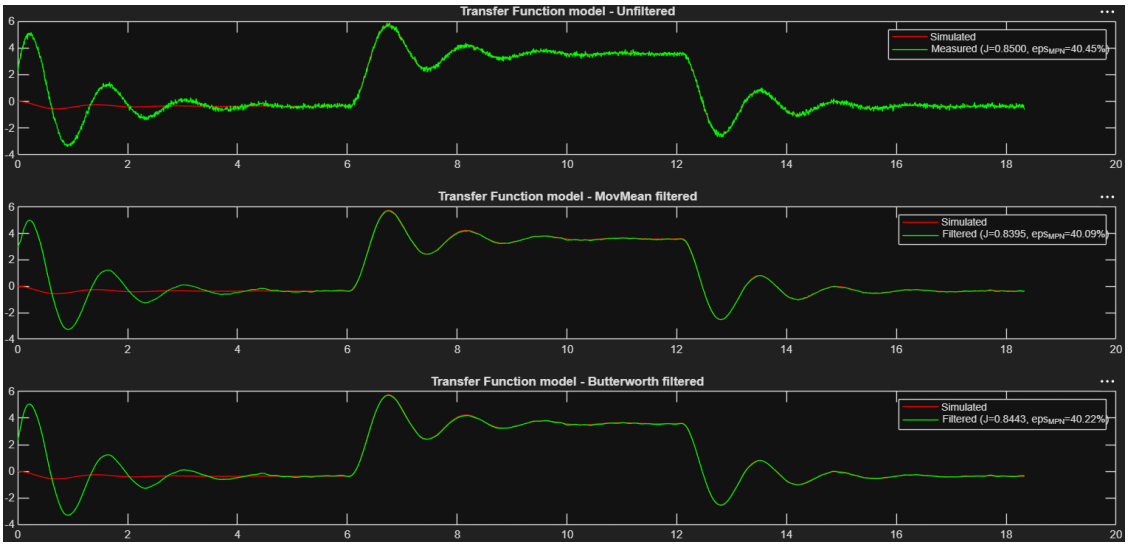


Figure 5 — Validation of the transfer function model.

Initially, the simulated signal oscillated faster than the output signal, which suggested that ωn was larger than it should have been. It was reduced by increasing the oscillation period Tosc, by finely adjusting the indices chosen at the extremum points.

Appendix — MATLAB source code

The following code contains the complete implementation of the presented identification method: computation of the static gain, regression on the oscillation section, reconstruction of the model as a transfer function and in state space, signal filtering and computation of the performance indicators.

```
%% System identification

% Regresie formula 1 slide 21

i1 = 2461;
i2 = 2907;
i3 = 5482;
i4 = 5979;

u0  = mean(u(i1:i2));
y0  = mean(y(i1:i2));
ust = mean(u(i3:i4));
yst = mean(y(i3:i4));

K = (yst - y0)/ (ust - u0);

%%
tmin = 3033;
tmax = 6072;

y_aux = abs(y(tmin:tmax) - yst);
t_aux = t(tmin:tmax);
figure;
plot(t_aux, y_aux, 'r');

im1 = 6;
im2 = 343;
im3 = 710;
im4 = 1070;
im5 = 1402;

t_reg  = t_aux([im1 im2 im3 im4 im5]);
y_reg = log(y_aux([im1 im2 im3 im4 im5])) ;

figure;
plot(t_reg, y_reg, 'g');
title('Regression Line');

A_reg = [sum(t_reg.^2) sum(t_reg);
   sum(t_reg) length(t_reg)];
B_reg = [sum(t_reg.*y_reg);
   sum(y_reg)];
theta = A_reg\B_reg;
alpha = theta(1); %Re{s1,2}

Tosc = t(im3) - t(im1);
wosc = 2*pi/Tosc; % Im{s1,2}

zeta = - alpha/(sqrt(alpha.^2 + wosc.^2));
wn   = - alpha/zeta;

%% State Space
A = [ 0 1;
   -wn.^2 -2*zeta*wn];
B = [0;
   K*wn.^2];
C = [1 0];
D = 0;

sys = ss(A,B,C,D);
```

```
y_sim_ss = lsim(sys,u,t,[y(1) 24.49]);

%% Transform

H = tf(K*wn.^2, [1, 2*zeta*wn, wn.^2]);
y_sim_tf = lsim(H,u,t);

%% Filtering
idx_val = (tmax+1):length(t); % Unused values

% Moving average filter
windowSize = 55;
y_mov = movmean(y,windowSize);

% Butterworth filter
fs = 1/Ts;
fc = 5;
ord = 4;
[b_filt, a_filt] = butter(ord, fc/(fs/2), 'low');
y_butter = filtfilt(b_filt, a_filt, y);

%% Performance indexes
% J = ||eps||_2 / sqrt(N)
% eps_MPN = ||eps||_2 / ||y - mean(y)||_2

% State Space - nefiltrat
J_ss_full    = norm(y_sim_ss - y) / sqrt(length(y));
eMPN_ss_full = norm(y_sim_ss - y) / norm(y - mean(y));
J_ss_val     = norm(y_sim_ss(idx_val) - y(idx_val)) / sqrt(length(idx_val));
eMPN_ss_val  = norm(y_sim_ss(idx_val) - y(idx_val)) / norm(y(idx_val) - mean(y(idx_val)));

% State Space - movmean
J_ss_mov_full    = norm(y_sim_ss - y_mov) / sqrt(length(y_mov));
eMPN_ss_mov_full = norm(y_sim_ss - y_mov) / norm(y_mov - mean(y_mov));
J_ss_mov_val     = norm(y_sim_ss(idx_val) - y_mov(idx_val)) / sqrt(length(idx_val));
eMPN_ss_mov_val  = norm(y_sim_ss(idx_val) - y_mov(idx_val)) / norm(y_mov(idx_val) - mean(y_mov(idx_val)));

% State Space - Butterworth
J_ss_butter_full    = norm(y_sim_ss - y_butter) / sqrt(length(y_butter));
eMPN_ss_butter_full = norm(y_sim_ss - y_butter) / norm(y_butter - mean(y_butter));
J_ss_butter_val     = norm(y_sim_ss(idx_val) - y_butter(idx_val)) / sqrt(length(idx_val));
eMPN_ss_butter_val  = norm(y_sim_ss(idx_val) - y_butter(idx_val)) / norm(y_butter(idx_val) - mean(y_butter(idx_val)));

% Transfer Function - nefiltrat
J_tf_full    = norm(y_sim_tf - y) / sqrt(length(y));
eMPN_tf_full = norm(y_sim_tf - y) / norm(y - mean(y));
J_tf_val     = norm(y_sim_tf(idx_val) - y(idx_val)) / sqrt(length(idx_val));
eMPN_tf_val  = norm(y_sim_tf(idx_val) - y(idx_val)) / norm(y(idx_val) - mean(y(idx_val)));

% Transfer Function - movmean
J_tf_mov_full    = norm(y_sim_tf - y_mov) / sqrt(length(y_mov));
eMPN_tf_mov_full = norm(y_sim_tf - y_mov) / norm(y_mov - mean(y_mov));
J_tf_mov_val     = norm(y_sim_tf(idx_val) - y_mov(idx_val)) / sqrt(length(idx_val));
eMPN_tf_mov_val  = norm(y_sim_tf(idx_val) - y_mov(idx_val)) / norm(y_mov(idx_val) - mean(y_mov(idx_val)));

% Transfer Function - Butterworth
J_tf_butter_full    = norm(y_sim_tf - y_butter) / sqrt(length(y_butter));
eMPN_tf_butter_full = norm(y_sim_tf - y_butter) / norm(y_butter - mean(y_butter));
J_tf_butter_val     = norm(y_sim_tf(idx_val) - y_butter(idx_val)) / sqrt(length(idx_val));
eMPN_tf_butter_val  = norm(y_sim_tf(idx_val) - y_butter(idx_val)) / norm(y_butter(idx_val) - mean(y_butter(idx_val)));

%% Plots - State Space model
figure
subplot(3,1,1)
plot(t,y_sim_ss,'r', t, y, 'g');
title('State Space model - Unfiltered')
legend('Simulated', sprintf('Measured (J=%.4f, eps_{MPN}=%.2f%%)', J_ss_full, 100*eMPN_ss_full))

subplot(3,1,2)
plot(t,y_sim_ss,'r', t, y_mov, 'g');
```

```
title('State Space model - MovMean filtered')
legend('Simulated', sprintf('Filtered (J=%.4f, eps_{MPN}=%.2f%%)', J_ss_mov_full, 100*eMPN_ss_mov_full))

subplot(3,1,3)
plot(t,y_sim_ss,'r', t, y_butter, 'g');
title('State Space model - Butterworth filtered')
legend('Simulated', sprintf('Filtered (J=%.4f, eps_{MPN}=%.2f%%)', J_ss_butter_full, 100*eMPN_ss_butter_full))

%% Plots - Transfer Function model
figure
subplot(3,1,1)
plot(t,y_sim_tf,'r', t, y, 'g');
title('Transfer Function model - Unfiltered')
legend('Simulated', sprintf('Measured (J=%.4f, eps_{MPN}=%.2f%%)', J_tf_full, 100*eMPN_tf_full))

subplot(3,1,2)
plot(t,y_sim_tf,'r', t, y_mov, 'g');
title('Transfer Function model - MovMean filtered')
legend('Simulated', sprintf('Filtered (J=%.4f, eps_{MPN}=%.2f%%)', J_tf_mov_full, 100*eMPN_tf_mov_full))

subplot(3,1,3)
plot(t,y_sim_tf,'r', t, y_butter, 'g');
title('Transfer Function model - Butterworth filtered')
legend('Simulated', sprintf('Filtered (J=%.4f, eps_{MPN}=%.2f%%)', J_tf_butter_full, 100*eMPN_tf_butter_full))
```